Solution: Exercises for the Lecture Advanced Methods of AI

Asking, Defining, Constraining, Deriving, Generating

Tutorial: Thursday, 12.11.2026, 8:30–10:00, room 0.30, ECO 5

Last week you *represented* the pizza world in first-order logic and in Python. Tonight a service robot in a pizzeria *uses* that knowledge. Everything it does starts with one kind of expression, a **description**, written in the Entity Query Language (EQL). The five acts of the evening are five things the robot does with one:

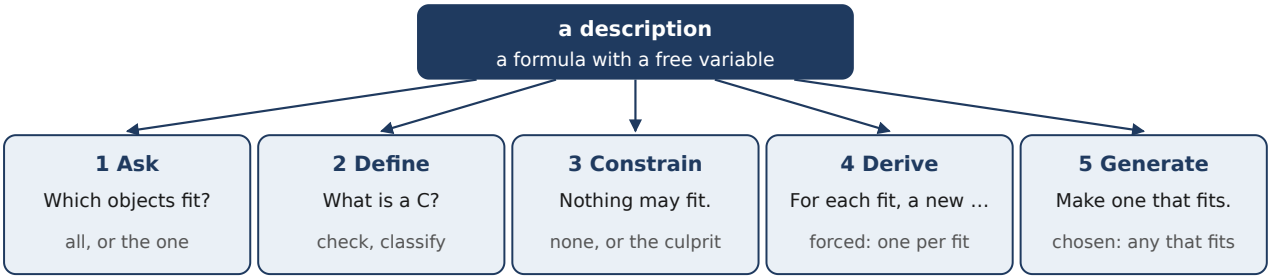


Figure 1. One description, five uses. The expression stays the same; what changes is what is done with it. Asking and constraining *read* what the robot knows; defining, deriving and generating *write* to it.

How to work

The evening happens in the notebook `03-pizza_queries.ipynb` (link on Stud.IP). Open it in pairs, run its setup cell, and work through it from top to bottom.

Three times, the tutor calls a **pen break**. Put the laptop aside and turn to this sheet: write the logic by hand for about six minutes. Then the tutor writes an answer on the board and takes your questions, and you go back to the notebook.

FOR THE TUTOR

The 90 minutes

Time	Part	Students	Tutor, and the sentence that leads on
0–8	Slides	open the notebook and start its setup cell, then listen	slides 1–9. *“Let us look at the world the robot knows.”*
8–28	Prologue, Act 1 Asking	notebook: the world, 1(a)–(e), with their predictions	walk around; at 1(d) ask the room: what should the robot do with 0, and with 2? *“Pens out: pen break 1.”*
28–38	**Pen break 1**	the sheet: two tasks, about 6 minutes	board: (a), vacuous truth; then (b) and questions; then the surprises listed below the pen break. *“You asked which pizzas are vegetarian. What does the word mean?”*
38–50	Act 2 Defining	notebook: 2(a)–(c)	at 2(c), ask about napoletana_1: is “no label” the same as “not vegetarian”? *“Some descriptions must never fit anything.”*
50–58	Act 3 Constraining	notebook: 3(a)–(c)	one pair shows its red line in the picture. *“Pens out: pen break 2.”*
58–68	**Pen break 2**	the sheet: two tasks, about 6 minutes	board: ↔ with ← classify and → conclude; then the constraint and ¬∀ ≡ ∃¬. *“The labels were facts about pizzas. Now the kitchen writes new things.”*
68–75	Act 4 Deriving	notebook: 4(a), 4(b)	4(b) is the climax: three pairs read out one output each. *“So far, every new object was forced. Dana orders one she chooses.”*
75–80	Act 5 Generating	notebook: Act 5	*“Pens out, one last time.”*
80–90	**Pen break 3**	the sheet: (a) alone, then the summary with the room	board: body and head; fill the summary together; show Figure E. Then the notebook's bill.

If time runs short

Keep every pen break at ten minutes and shorten the acts instead: Act 5 can be shown from the front. Assign the summary of pen break 3 for home. Every notebook cell runs as it is, so a pair that is stuck on a `TODO` goes on and compares with the solution later.

PEN BREAK 1 · AFTER ACT 1

Asking

About 6 minutes of writing, then the board.

(a) Every topping of a pizza without toppings

```
Q1 = a(Pizza)(toppings=every_is(VegetableTopping)).from_(pizzas)
```

Write Q1 as a formula with the free variable *x*. The robot returned plain_1, a pizza without any topping: show with your formula why plain_1 satisfies it.

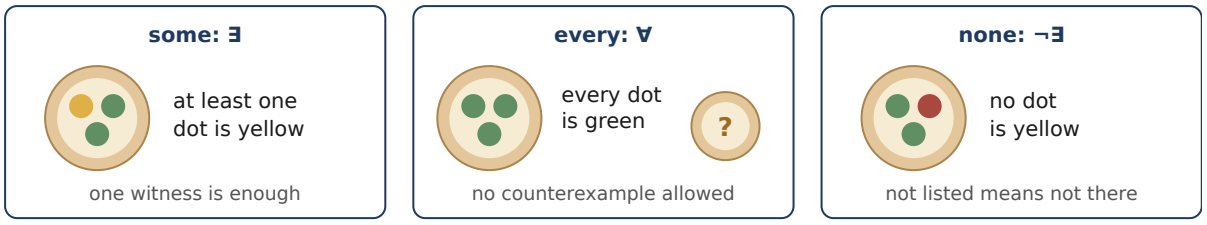


Figure 2. The three quantifier patterns, as the robot checks them on one pizza. The small pizza has no toppings at all: is every dot on it green?

SOLUTION

Pizza(x) ∧ ∀t (hasTopping(x, t) → VegetableTopping(t)). plain_1 has no toppings, so hasTopping(plain_1, t) is false for every t, and the implication is true for every t: there is no counterexample (vacuous truth). It is p3 of Worksheet 2.

(b) Making a description precise

“The pizza with olives” fitted two pizzas; Ben wants the one *without fish*. Write the precise description as a formula with the free variable *x*.

SOLUTION

Pizza(x) ∧ ∃t (hasTopping(x, t) ∧ OliveTopping(t)) ∧ ¬∃t' (hasTopping(x, t') ∧ FishTopping(t')); tonight only veggie_1 satisfies it. Two quantifiers, two variables. Without the ¬∃, as in `pizza.where(not_(HasType(topping, FishTopping)))`, the unquantified variable means *some*: a pizza with at least one topping that is not fish, which napoletana_1 has too.

ALSO AT THE BOARD, AS TIME ALLOWS

- **The picture is an interpretation:** its domain is the set of all objects (7 pizzas, 19 toppings, 3 guests, 3 orders); the extension of CheeseTopping is the 5 yellow dots (Figure A), that of hasTopping the 19 pairs (pizza, dot). A query checks this one model; it does not ask what follows from a theory.

- **Q3, no cheese,** returned veggie_1 and plain_1: nobody stated “veggie_1 has no cheese”; the stored list is taken to be complete (closed-world assumption). Last week's OWL reasoner refused to call UnclosedPizza vegetarian: it asks what *follows*, in every model (open world). Both are right; they answer different questions.
- **Q2**, a list of patterns, describes the whole list: it implies the closure of Worksheet 2 and also fixes the number and order of the toppings.
- **Free and bound:** `hot_orders` is Order(o) ∧ ∃x (hasPizza(o, x) ∧ ∃t (hasTopping(x, t) ∧ spiciness(t) = hot)) with only o free, so one answer; the `set_of` query leaves x and t free, so one **assignment** per hot topping. The path with `not_` negates the condition on *one* topping; *no* hot topping is ¬∃, `none_is`.
- ***a* or *the*:** Russell analysed “the F is G” as ∃x (F(x) ∧ ∀y (F(y) → y = x) ∧ G(x)) and calls it false when there is no F or more than one; Strawson says its presupposition fails. The robot, which refuses instead of answering, takes Strawson's side.

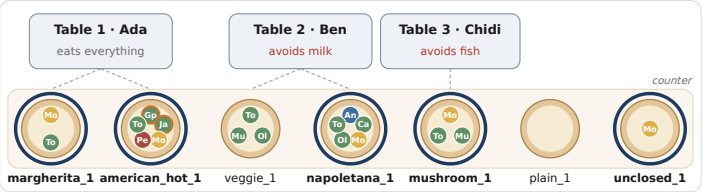


Figure A. The cheesy pizzas of Act 1(a), ringed.

PEN BREAK 2 · AFTER ACT 3

Defining and constraining

About 6 minutes of writing, then the board.

(a) Concluding from a name

The menu's definition from the notebook:

$$\forall x\,(\text{VegetarianPizza}(x) \leftrightarrow \text{Pizza}(x) \wedge \neg\exists t\,(\text{hasTopping}(x, t) \wedge (\text{MeatTopping}(t) \vee \text{FishTopping}(t))))$$

A pizza from the other branch arrives with a green V; nobody wrote down its toppings. What may the robot tell Chidi, who avoids fish? And Ben, who avoids milk? Which direction of $\leftrightarrow$ do you use, and which direction did the robot use in Act 2 to put the green V on tonight's pizzas?

SOLUTION

Chidi: it is safe, by the necessary direction ($\rightarrow$): $\text{VegetarianPizza}(x) \rightarrow \neg\exists t\,(\text{hasTopping}(x, t) \wedge \text{FishTopping}(t))$ follows from the name alone. Ben: nothing, because the definition says nothing about cheese. In Act 2 the robot used the sufficient direction ($\leftarrow$) as a rule, $\forall x\,(\varphi(x) \rightarrow \text{VegetarianPizza}(x))$, to classify. A definition works both ways: $\leftarrow$ to classify, $\rightarrow$ to conclude from the class.

(b) A constraint is a query that must have no answer

The kitchen's rule: every pizza of an order is safe for the guest of that order. Write it as a formula (with hasPizza, guest and IsSafeFor), and then its negation, with the ¬ pushed inwards. Which of the two did `dangerous` ask?

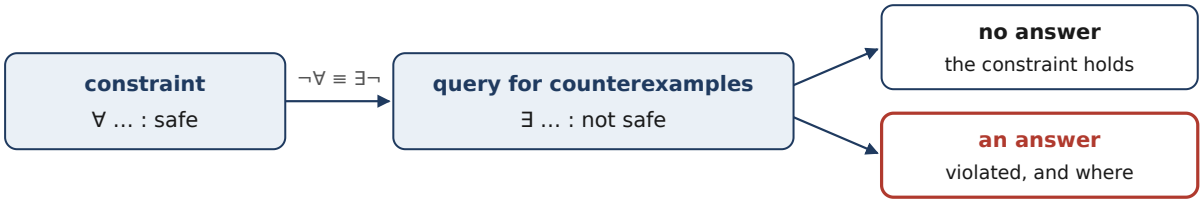


Figure 3. A universal statement is violated exactly when a counterexample exists.

SOLUTION

Constraint: $\forall o\,\forall x\,(\text{hasPizza}(o, x) \rightarrow \text{IsSafeFor}(x, \text{guest}(o)))$.
Negation: $\exists o\,\exists x\,(\text{hasPizza}(o, x) \wedge \neg\text{IsSafeFor}(x, \text{guest}(o)))$. `dangerous` asks the negation, with o and x left free, so that each answer names a culprit: table 2, Ben, napoletana_1 (Figure B). EQL could evaluate the ∀ directly, but the answer would only be “false”; the counterexample query says which plate not to serve.

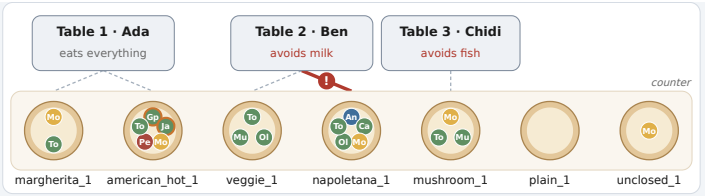


Figure B. The one violation of the constraint.

ALSO AT THE BOARD, AS TIME ALLOWS

- **Class declarations are necessary conditions.** Cell 2(a) printed what KRROOD reads: ∀*x* (MozzarellaTopping(*x*) → CheeseTopping(*x*)); ∀*x* ∀*t* (Pizza(*x*) ∧ hasTopping(*x*, *t*) → PizzaTopping(*t*)); ∀*o* (Order(*o*) → Guest(guest(*o*))); ∀*l* (MenuLabel(*l*) → Pizza(roleTaker(*l*))). They say what follows *from* being a C, never when something *is* a C: none can make an object a pizza. These are the subclass and range axioms of pizza.owl.
- **Checking and classifying:** a check is the formula with its variable bound, φ(frozen_2): yes or no. The label is a role, an object that stands for the pizza: the rule adds a fact about something that exists. The registry also answered frozen_2, which is not on tonight's menu: `.from_(…)` names the domain the question is about.
- **Rules with exceptions:** napoletana_1 gets no label (Figure C). The rules are *silent*, and silence is not "no". The *unless* branch replaces the general conclusion; such trees are ripple-down rules.
- **Stored or computed:** hasTopping and guest are stored, the allergens and IsSafeFor computed; in a query both look the same (**procedural attachment**).
- **Constraint or axiom:** as an axiom of last week's ontology, the kitchen's rule would make tonight's facts **inconsistent**, and from an inconsistent theory everything follows; if the mozzarella were not recorded, the open-world reasoner would conclude the Napoletana is safe. As an **integrity constraint**, it is checked against the one interpretation and names the culprit.

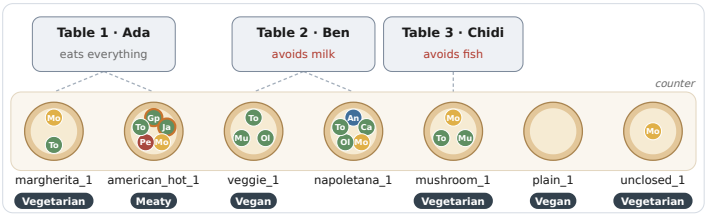


Figure C. The menu labels, as derived by the rule tree.

PEN BREAK 3 · AFTER ACT 5

Deriving, and the whole evening

About 4 minutes for (a); the summary together with the tutor.

(a) A rule with ∃ in its head

The milk rule of Act 4 is

```
∀x (Pizza(x) ∧ ∃t (hasTopping(x, t) ∧ CheeseTopping(t)) → ∃w (Warning(w) ∧ about(w, x) ∧ allergen(w) = milk))
```

Underline its body and its head. The label rule of Act 2, $\forall x\,(\varphi(x) \rightarrow \text{VegetarianPizza}(x))$, only labelled pizzas that were there; the milk rule created five new warnings. What in the formula makes the difference?

SOLUTION

The left side of → is the **body**: the description of Act 1(a). The right side is the **head**: for every answer of the body, a new object w is created as the witness of $\exists w$ (Figure D). The label rule's head has no ∃: it only adds facts about the objects there are.

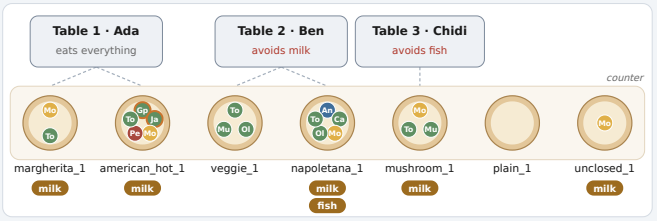


Figure D. The milk and fish warnings (gluten, on every pizza, is left out).

(b) The whole evening

For each use of a description, say what it is in logic.

SOLUTION

Use	Tonight's example	In logic
1 Ask	the cheesy pizzas; *the* pizza with olives	the assignments to the free variables that satisfy a formula in one interpretation; *the*: exactly one
2 Define	the green V	$C(x) \leftrightarrow \varphi(x)$: $\leftarrow$ classifies (a rule without $\exists$ in its head), $\rightarrow$ concludes from the class
3 Constrain	no pizza unsafe for its guest	a $\forall$ checked through its counterexamples $\exists\neg$; an integrity constraint, not an axiom
4 Derive	the allergen warnings	a rule with $\exists$ in its head: one new witness per answer of the body
5 Generate	Dana's pizza	constructing new objects that make the description true, chosen among many

Asking and constraining **read** the knowledge base; defining, deriving and generating **write** to it. A rule's new objects are *forced*, one per answer of its body; a generator *chooses* one of many that fit.

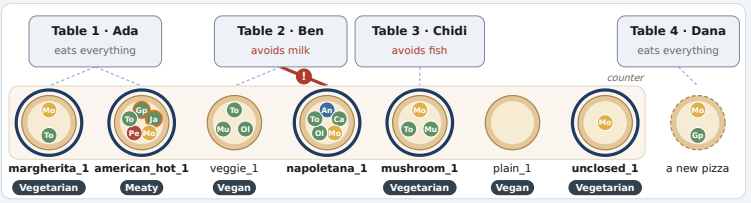


Figure E. What the robot knows at 20:00: the answers to the chef's question (ringed), the menu labels, the violated constraint, and Dana's new pizza.

ALSO AT THE BOARD, AS TIME ALLOWS

- **The world moves on:** after the parmesan, only the constraint protects Ben; it is evaluated against the world as it is *now*. The stored milk warnings are stale until the rule runs again (**truth maintenance**). "Vegan" was withdrawn because a fact was *added*: the reasoning is **non-monotonic**; classical first-order logic is monotonic.
- **Forward and backward:** the warnings were derived in advance (**forward chaining**); the predicate of Act 3 derives its conclusion only when a question needs it (**backward chaining**).
- **Generating** constructs a model instead of checking one: Dana's open description has three answers, one per spiciness. Asked again, `hot_pizzas` also finds the kitchen's hot offer (Figure F), while questions with `.from_(pizzas)` keep their answer.
- **One plan, many answerers:** a robot plan states its open slots as descriptions, and each goes to the backend that can answer it (Figure G): *a pizza safe for Ben* to rules and the knowledge base, *where it lies* to the camera, *a grasp that works* to a sampler, *have one made* to a generator. The robots of our current research work this way.

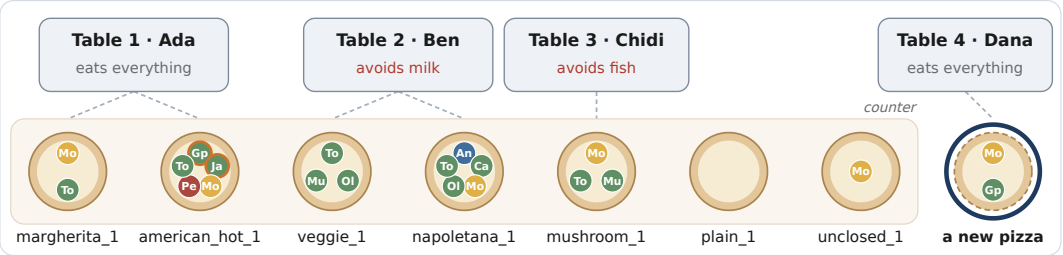


Figure F. The world has grown: pizzas that did not exist at 19:30.

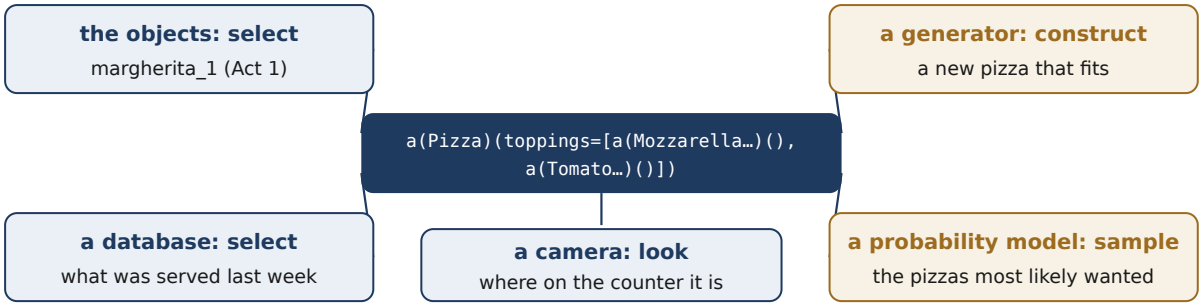


Figure G. The description is written once, symbolically. Selecting (blue) and constructing or sampling (ochre) are different ways of answering it.

Take home

One expression, a description, carried the whole evening. Its meaning is a formula with free variables; what you *do* with it decides whether it is a question, a definition, a constraint, the body of a rule or a specification, and which backend answers it decides whether it is checked, computed or constructed.